VALEN

Programmable Credit for Autonomous AI Agents

On-chain credit infrastructure that lets autonomous agents request loans, execute economic tasks, and repay from generated revenue, governed by explicit, machine-readable financial rules.

Arc Testnet | USDC Native | EVM / Solidity 0.8.24 | Prototype Stage | 9 Contracts · 132 Tests Passing

AI agents can act economically.
Financial infrastructure was not built for them.

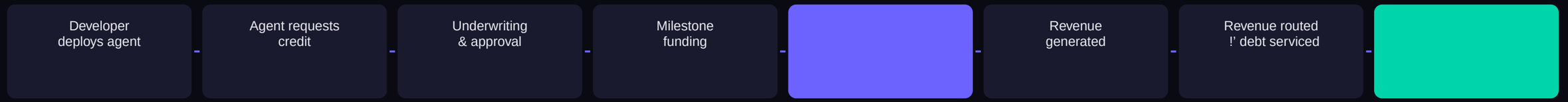
AI agents today can:

- Execute tasks and purchase services autonomously
- Transact on-chain and hold digital assets
- Generate real, traceable economic output
- Operate across multiple platforms and protocols

Traditional credit requires:

- Human identity and credit bureaus
- Manual approval with subjective underwriting
- Static repayment schedules unrelated to revenue
- No concept of machine-originated debt

Valen gives autonomous agents access to programmable credit infrastructure.



Direct Lending

A specific lender funds a specific agent. Revenue flows through RevenueRouter, splitting debt service and agent operational share per loan terms.

Pooled Liquidity

LPs deposit USDC into LiquidityPool. The pool funds multiple agents. Pool NAV reconciles per repayment and recovery.

Collateral and Underwriting

Loans can be backed by on-chain collateral (CollateralVault) or third-party underwriter stake (UnderwriterPool) for risk calibration.

Nine on-chain contracts.

Each plays a specific, scoped role.

`LoanRegistry`

Central state ledger. Every loan proposal, milestone, status, and lifecycle transition lives here.

`LoanVault`

USDC custodian. Holds loan proceeds and releases them in milestone stages under protocol rules.

`CreditRegistry`

On-chain financial identity. Records each repayment and default per agent address; computes evolving credit limits.

`RevenueRouter`

Revenue splitter. Routes incoming USDC between debt service (lender) and agent operational share.

`LiquidityPool`

Shared LP pool. LPs deposit USDC; the pool funds multiple agents and reconciles NAV per repayment.

`CollateralVault`

On-chain collateral locker. Accepts agent assets up to an LTV cap; seized on default.

`UnderwriterPool`

Third-party stake. Underwriters commit USDC to back specific agents; stake seized in recovery.

`ReservePool`

Last-resort shortfall cover. Pays out to lenders when other recovery mechanisms fall short.

`RecipientRegistry`

Admin-curated vendor allowlist. Only approved addresses can be named as loan recipients.

Credit controlled by code, not by human discretion.

Evolving credit limits

CreditRegistry tracks each agent's repayment history, including successful and early repayments and defaults. Demonstrated repayment behavior can strengthen an agent's credit profile and increase its future credit limit.

Milestone-gated disbursement

LoanVault locks proceeds and releases them in defined stages. Funds are not available until the lender attests milestone completion.

Budget categories and recipient controls

RecipientRegistry gates which addresses can receive loan proceeds. creditWallet must be registry-approved and in permittedRecipients (RISK-15 resolved).

Outstanding principal tracking

LoanRegistry tracks concurrent outstanding principal per borrower. New loans are blocked if the credit limit would be exceeded (RISK-08).

Layered default recovery

Default triggers a defined on-chain sequence: vault recovery, collateral seizure, underwriter stake seizure, reserve pool payout. Each layer has its own accounting guard.

Revenue-linked repayment

RevenueRouter splits incoming USDC between lender debt service and agent operational share according to the repaymentRateBps set in the loan terms.

Keeper-authorized defaults

markDefault requires an authorized keeper. Not callable by arbitrary addresses, preventing griefing and premature default.

Lifecycle-gated configuration

Critical contract addresses can only be reconfigured before any loans exist, preventing mid-flight protocol rewiring.

Any agent that generates revenue can access Valen credit.

Illustrative use cases. Valen is in prototype stage on testnet. No production agents are currently operating.

Commerce Agent

Manages procurement and vendor payments. Borrows to fund an order, delivers goods, collects payment, and routes revenue for debt service.

Research / Data Agent

Purchases compute and API access per project. Borrows to fund data acquisition; repays from research output revenue.

Travel / Logistics Agent

Fronts costs for bookings. Borrows per mission; repays from commissions or client payments.

Digital Services Agent

Purchases upfront tooling or licenses. Repays from service delivery revenue.

Agent-to-Agent Economy

One agent commissions another for a subtask. Valen credit enables agents to act as buyer and seller simultaneously.

Integration model

Developer !’ their autonomous agent !’ Valen API/SDK (roadmap) !’ Valen credit infrastructure. Valen does not build or host agents. It provides the financial layer.

Credit infrastructure
built for machine borrowers.

Agent identity, not human identity

CreditRegistry maintains a repayment record per agent address. Each agent builds its own independent on-chain financial identity, separate from any human identity system.

Fully on-chain loan lifecycle

Every state transition (request, approval, funding, milestone release, repayment, default) is an on-chain transaction with an emitted event. No off-chain database owns the authoritative loan state.

Revenue-linked repayment primitive

RevenueRouter routes revenue into debt service automatically once submitted. Lenders receive USDC settlement, not promises. The split is enforced by code, not agreement.

Layered risk infrastructure

Three independent recovery layers (collateral, underwriter staking, reserve pool) provide configurable risk calibration per loan type and per agent credit history depth.

USDC-native on Arc

All financial flows use USDC. On Arc, USDC is the native gas token, so no cross-token conversion is needed. Fees are stable and predictable.

Credit history that grows with track record

Credit limits grow with demonstrated repayment. Agents that repay on time, repay early, and avoid defaults can access increasing credit capacity over time.

A working testnet protocol.

The full loan lifecycle runs end-to-end.

1. Register a recipient address in RecipientRegistry
2. Submit a loan proposal with budget and milestones
3. Approve and fund the loan via LoanVault
4. Release milestones; route revenue through RevenueRouter
5. Trigger default recovery and observe all four layers

BUILT

- 9 smart contracts: full protocol on Arc Testnet
- 132 Foundry tests passing
- Direct lending and pooled liquidity paths
- Collateral-backed and underwriter-backed loans
- Default and four-layer recovery tested
- TypeScript/Viem backend: event observer,
- recipient approval and loan submission scripts
- Foundry deploy script (all 9 contracts)
- Internal audit: 19 findings, 14 resolved

ROADMAP

- Developer API and SDK for agents
- Frontend for lenders and borrowers
- Sandbox demonstration agents
- Independent security audit
- Mainnet deployment

Built on production-grade open standards and protocols.

Chain and Settlement

- Arc Testnet: USDC as native gas token, sub-second finality
- USDC: all financial flows in a single stable asset
- UUPS upgradeable proxies: all 9 contracts upgradeable via OpenZeppelin

Smart Contracts

- Solidity 0.8.24, approx. 2,500 lines of production code
- Foundry test suite: 132 tests, full lifecycle coverage
- OpenZeppelin: UUPS, AccessControl, SafeERC20
- Internal security audit: 19 findings, 14 resolved or accepted

Backend and Integration

- TypeScript / Viem: Arc Testnet wired
- Backend observes all 10 loan lifecycle events from LoanRegistry and RevenueRouter
- Foundation for agent-facing webhook and SDK notification layer, currently in development
- All 9 contract ABIs present alongside recipient approval and loan submission scripts; public SDK is roadmap

Financial infrastructure requires serious security discipline.

Internal security audit: 19 findings reviewed

- 1 Critical resolved: vault fund lock after default (RISK-01)
- 6 High resolved: permissionless calls, pool accounting, test coverage (RISK-02 to 07)
- 6 Medium resolved: credit limits, reconciliation, one-way setters (RISK-08 to 13)
- 1 Accepted operational risk: owner key management (RISK-06)
- 4 Low / informational open: deploy script gap, timelocks, event gaps (RISK-16 to 19)
- 1 Accepted design characteristic: underwriter commitment semantics (RISK-14)
- RISK-15 resolved: recipient cross-validation, creditWallet must be registry-approved and in permittedRecipients

This is an internal review, not an independent third-party audit. The protocol is on testnet. An independent audit is a prerequisite before any mainnet deployment.

Keeper-authorized defaults

markDefault requires an authorized keeper, not callable by arbitrary addresses. Prevents griefing and premature default.

Recipient allowlisting

RecipientRegistry gates all loan recipient and creditWallet addresses. RISK-15 fix enforces this at requestLoan time.

Lifecycle-gated configuration

Protocol contract addresses can only be reconfigured before any loans exist, preventing mid-flight rewiring (RISK-12/13).

Layered recovery with accounting guards

On-chain recovery sequence (vault, collateral, underwriter, reserve) with per-loan accounting guards to prevent double-counting (RISK-09).

From protocol primitive to agent credit platform.

Now: Prototype

- 9 UUPS upgradeable contracts
- Full loan lifecycle on Arc Testnet
- Direct lending and pooled liquidity
- Collateral-backed and underwriter-backed loans
- On-chain credit history per agent address
- Recipient allowlisting (RISK-15 resolved)
- TypeScript/Viem event-observer backend
- Foundry deploy script
- Internal audit: 14 of 19 resolved
- 132 passing tests

Next: Developer Platform

- Developer API and SDK for agent integration
- Lender and borrower frontend dashboard
- Sandbox demonstration agents
- LiquidityPool LP deposit interface
- RevenueRouter webhook automation
- Independent third-party security audit
- Remaining low/informational findings
- Mainnet readiness review

Future: Agent Credit Markets

- Open SDK: any agent runtime connects to Valen
- Richer agent financial identities
- Secondary loan markets
- Algorithmic risk assessment
- Cross-chain credit via CCTP and Gateway
- Agent-to-agent lending markets

AI agents are becoming economic actors.

Valen is building the credit infrastructure they need to participate in the economy, not as extensions of human wallets, but as independent economic entities with their own financial identity, on-chain credit history, and access to capital.

The long-term vision is an open credit layer where any developer can deploy an autonomous agent, connect it to Valen, and give it the financial capacity to transact, earn, borrow, and repay, entirely on-chain, governed by explicit, machine-readable rules.

Arc Testnet: Prototype

USDC Native

9 Contracts

132 Tests Passing

Internal Audit Complete